

Распределенная сеть нейронных сетей на микроконтроллерах ESP32

Исследовательский отчет и проектная проработка дипломной темы

Подготовлено для дипломного исследования

2026-02-16

Содержание

1	1. Как читать этот документ	4
2	2. Executive Summary (обновленная версия)	4
3	3. Литературный обзор	4
3.1	3.1 База TinyML на MCU	4
3.2	3.2 Квантование, дистилляция, pruning как научная основа	5
3.3	3.3 Распределенный и split инференс	5
3.4	3.4 Бенчмарки и reproducibility	5
3.5	3.5 Аннотированный обзор ключевых статей (что именно брать в дипломный текст)	6
3.5.1	[R2] MCUNet	6
3.5.2	[R3] MCUNetV2	6
3.5.3	[R11] NoNN	6
3.5.4	[R10] Edgent / co-inference	6
3.5.5	[R12] Split-Learning TinyML on ESP32-S3	7
3.5.6	[R6] Integer-only quantization	7
3.5.7	[R7] Distillation	7
3.5.8	[R9] KWS pruning/quantization on low-power MCU	7
3.6	3.6 Литературные пробелы (где есть место для твоей новизны)	7
4	4. ESP32/ESP32-S3 для ML: тех-обзор «для чайника, но без упрощенчества»	8
4.1	4.1 Что важно понять сразу	8
4.2	4.2 ESP32 семейство (практический взгляд)	8
4.2.1	ESP32 (классический)	8
4.2.2	ESP32-S3 (предпочтительный для диплома)	8
4.2.3	ESP32-C3 (один core, RISC-V)	8
4.3	4.3 Память: внутренняя RAM vs PSRAM	8
4.4	4.4 RTOS и многозадачность	8
4.5	4.5 Сеть и задержки: ESP-NOW/Wi-Fi	8
4.6	4.6 Почему задержки в дипломе критичны	10
4.7	4.7 Мини-курс по микроконтроллерам для ML-специалиста	10
4.7.1	Что такое MCU на практике	10
4.7.2	Как устроен жизненный цикл программы на ESP32	10
4.7.3	Что такое «ядра и пиннинг» в практическом смысле	10
4.7.4	Что такое latency budget	10
4.7.5	Почему offline accuracy может обмануть	10
4.7.6	Что такое «безопасный» путь к работающему прототипу	11
4.8	4.8 Практические ограничения ESP-NOW именно для ML-пакетов	11
4.9	4.9 Power-save и «скрытые» задержки	11
5	5. Твоя исходная идея («блоки сети на разных MCU») и ее критический разбор	11
5.1	5.1 В чем сильная сторона идеи	11
5.2	5.2 Главный риск: стоимость передачи активаций	11
5.3	5.3 Когда блочная идея все же жизнеспособна	12

6	5.4 Как спасти исходную «блочную» идею и сделать ее научно сильной	12
7	5.5 Универсальный план эксперимента для CC-NBG (не только под аудио)	14
8	6. Кандидаты дипломных концептов (с критикой)	14
8.1	6.1 Кандидат А (рекомендуемый): CC-NBG как базовая архитектура, KWS как первый демонстратор	14
8.2	6.2 Кандидат В: Распределенное обнаружение бытовых аномалий (звук + вибрация + газ)	14
8.3	6.3 Кандидат С: Чистый layer-split DNN между ESP32	15
8.4	6.4 Кандидат D: Федеративная персонализация tiny-моделей на ESP32	15
8.5	6.5 Сравнительная матрица (формализованный выбор)	15
9	7. Лучшая идея: детальная проработка	16
9.1	7.1 Постановка задачи	16
9.2	7.2 Архитектура системы	16
9.2.1	Узлы (Node-i)	16
9.2.2	Координатор	16
9.2.3	Почему это communication-efficient	16
9.3	7.3 Формальная математическая модель	16
9.4	7.4 Routing policy (ядро новизны)	17
9.5	7.5 Почему это лучше для ESP32 именно с учетом задержек	17
9.5.1	Сравнение коммуникационной нагрузки	17
9.5.2	Реальный jitter	18
9.6	7.6 Предлагаемая двухуровневая архитектура (детально)	18
9.7	7.7 Как обучать под distributed режим, а не просто «обычный KWS»	18
9.8	7.8 Анти-гипотезы, которые нужно честно проверять	18
9.9	7.9 Memory budget: как быстро оценивать влезаемость модели	19
9.10	7.10 Почему ESP32-S3 предпочтительнее классического ESP32 для этой задачи	19
10	8. Экспериментальный дизайн (чтобы защита была сильной)	19
10.1	8.1 Аппаратный стенд	19
10.2	8.2 Программный стек	19
10.3	8.3 Данные	19
10.4	8.4 Бейзлайны и абляции	19
10.5	8.5 Метрики	20
10.6	8.6 Статистика	20
10.7	8.7 Протокол сценариев (чтобы результаты были валидны)	20
10.8	8.8 Как измерять энергию корректно	20
10.9	8.9 KPI для итоговой защиты	20
11	9. Инженерные детали реализации	21
11.1	9.1 Рекомендованная структура задач FreeRTOS	21
11.2	9.2 Формат пакета ESP-NOW	21
11.3	9.3 Синхронизация времени	21
11.4	9.4 Обработка потерь пакетов	21
11.5	9.5 Рекомендуемая структура репозитория	21
11.6	9.6 Минимальный протокол логирования	22
11.7	9.7 Типичные embedded-баги в таких проектах	22
12	10. Кодовый скелет (концептуально)	22
13	11. Где именно научная новизна (чтобы комиссия не сказала «это просто инженерка»)	23
13.1	11.1 Novelty 1: Communication-aware distillation	23
13.2	11.2 Novelty 2: Latency-aware adaptive routing	23
13.3	11.3 Novelty 3: Robust fusion under node/link failures	23
13.4	11.4 Как формально сформулировать вклад в ВКР	23
14	12. Слабые места проекта (критика заранее)	23
14.1	12.1 Риск синхронизации	23
14.2	12.2 Риск переусложнения	24
14.3	12.3 Риск недоказанной пользы	24
14.4	12.4 Риск «сетевое узкое место»	24

15	13. Почему этот диплом реально может быть «лучше существующих решений»	24
15.1	13.1 Пример ожидаемой таблицы результатов (шаблон)	24
15.2	13.2 Что считать «успехом диплома»	25
16	14. Дорожная карта диплома (пример на 16 недель)	25
17	15. Практические рекомендации «что делать завтра»	25
18	16. Заключение	25
19	Приложение А. Быстрый глоссарий (ML -> embedded)	26
20	Приложение В. Источники и ссылки	26
20.1	B.1 TinyML, distributed inference, optimization	26
20.2	B.2 ESP32 official documentation and tooling	27
20.3	B.3 Additional ecosystem references	28
21	Приложение С. Что еще можно добавить в финальную версию диплома	28
22	Приложение D. Расширенная аннотированная библиография	28
22.1	D.1 Системные и runtime-работы	28
22.1.1	[R1] CMSIS-NN	28
22.1.2	[R2] MCUNet	28
22.1.3	[R3] MCUNetV2	28
22.1.4	[R5] MEMA	28
22.1.5	[R4] On-device training under 256KB	29
22.2	D.2 Компрессия, квантование, дистилляция	29
22.2.1	[R6] Integer-only quantization	29
22.2.2	[R7] Distillation	29
22.2.3	[R8] Low precision quantization for TinyML	29
22.2.4	[R9] KWS pruning + quantization на low-power MCU	29
22.3	D.3 Distributed/split inference	29
22.3.1	[R10] Edgent	29
22.3.2	[R11] NoNN	29
22.3.3	[R12] Split-Learning TinyML on ultra-low-power nodes	29
22.4	D.4 Бенчмарки и датасеты	30
22.4.1	[R13] MLPerf Tiny	30
22.4.2	[R24] Speech Commands	30
22.4.3	[R31] Temporal Lambda Networks for KWS	30
22.4.4	[R32] MobileNets	30
22.5	D.5 Официальные источники Espressif (обязательны в дипломе)	30
22.5.1	[R21] ESP-NOW API	30
22.5.2	[R22] Wi-Fi performance/power-save	30
22.5.3	[R20] FreeRTOS SMP в ESP-IDF	30
22.5.4	[R19] External RAM / PSRAM	30
22.5.5	[R16], [R17], [R23], [R25]	30
23	Приложение Е. Сильные и слабые аргументы на защите	31
23.1	E.1 Сильные аргументы	31
23.2	E.2 Слабые аргументы (которые лучше не использовать)	31
23.3	E.3 Что спросит комиссия и как отвечать	31
24	Приложение F. Развернутые обзоры статей (сплошной текст)	31
24.1	F.1 TinyML и MCU-рантаймы	31
24.2	F.2 Компрессия, квантование и перенос знаний	32
24.3	F.3 Distributed и split inference: эволюция идей	32
24.4	F.4 Бенчмарки, датасеты и прикладные ориентиры	33
25	Приложение G. Финальная архитектурная позиция после критики	34
25.1	G.1 Конкретный критерий жизнеспособности split с энкодером/декодером	34
25.2	G.2 Практическое решение для диплома: гибридный двухконтурный режим	34
25.3	G.3 Что именно будет научной новизной в такой финальной версии	35

1 1. Как читать этот документ

Этот текст сделан как «книга-черновик» для диплома: с обзором литературы, инженерными ограничениями ESP32, сравнением проектных идей, критикой слабых мест и детальной проработкой одной сильной концепции.

Что здесь есть:

1. Литературный обзор по TinyML, распределенному инференсу, split/distributed подходам и оптимизации моделей.
2. Тех-обзор ESP32/ESP32-S3 «для человека из ML, но не из embedded».
3. Набор дипломных концептов и их критический разбор.
4. Выбор лучшей идеи под тему диплома.
5. Математическая постановка, алгоритм, архитектура системы, план экспериментов, метрики, кодовый скелет.
6. Подробный список источников с ссылками.

2 2. Executive Summary (обновленная версия)

В этой редакции центральная идея сформулирована не как «акустический проект», а как **универсальная распределенная нейросетевая система на кластере микроконтроллеров ESP32**. Акустика остается полезным и доступным демонстратором, но не является единственным смыслом работы.

Предлагаемая рамка называется здесь *distributed neural fabric*. В ней каждый микроконтроллер хранит и выполняет определенный набор нейросетевых блоков, а вся система работает как граф вычислений с ограничениями по памяти, трафику и задержке. Ключевой акцент смещается с «одной модели на одном устройстве» на задачу совместного вычисления: какие блоки где размещать, в каком виде передавать промежуточные представления и как динамически менять маршрут инференса при изменении канала связи или загрузки узлов.

Твоя исходная идея о разнесении блоков сети по разным ESP32 не является плохой. Ее слабость не в концепции, а в наивной реализации, когда между узлами гоняются крупные промежуточные тензоры без специально обученных bottleneck-представлений. Если же вставить в точки разреза обучаемые компрессоры, явно ограничить размер токенов связи, добавить routing policy с учетом текущей задержки и подготовить модель через communication-aware distillation, идея превращается в полноценную научную задачу с сильной новизной.

С практической точки зрения это реализуемо на бюджетном стенде, а с научной точки зрения дает четкие проверяемые гипотезы: насколько снижается трафик, как меняется p95/p99 задержка, где проходит граница, после которой layer-split становится невыгодным, и в каких режимах гибридная схема (block split + поздний fusion) выигрывает у baseline-решений.

3 3. Литературный обзор

3.1 3.1 База TinyML на MCU

Ключевая проблема TinyML: жесткие лимиты памяти, энергии и пропускной способности, при этом нужно сохранить точность и стабильную задержку.

Опорные работы:

1. **CMSIS-NN** (ядра для Cortex-M): классическая демонстрация, что библиотечные оптимизации дают большой прирост эффективности на микроконтроллерах [R1].
2. **MCUNet**: совместный поиск архитектуры модели + runtime (TinyNAS + TinyEngine), что критично именно для MCU, а не для смартфонов [R2].
3. **MCUNetV2**: patch-based inference для снятия memory bottleneck ранних слоев [R3].
4. **On-Device Training Under 256KB**: показывает направление к адаптации модели на устройстве, но с очень жесткой инженерной дисциплиной [R4].
5. **MEMA Runtime**: фокус на минимизации внешних memory accesses, то есть на реальных аппаратных узких местах [R5].

Вывод для диплома:

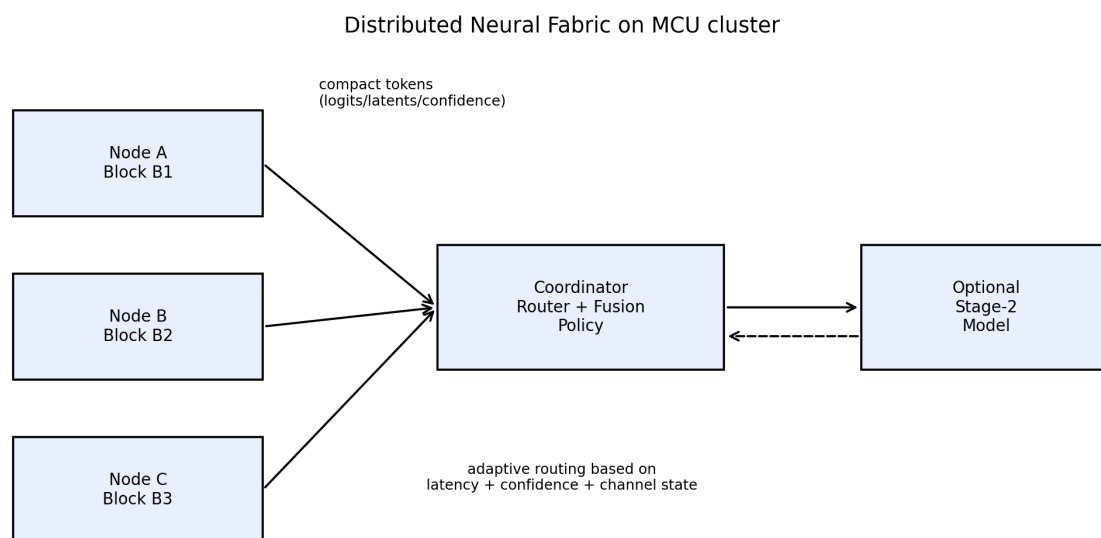


Рис. 1: Концепт универсальной архитектуры distributed neural fabric для кластера микроконтроллеров. Иллюстрация подготовлена в рамках этого отчета.

1. Нельзя рассматривать только архитектуру сети без рантайма и памяти.
2. В embedded-мире модель и системная реализация неразделимы.

3.2 3.2 Квантование, дистилляция, pruning как научная основа

1. **Integer-only quantization** (Jacob et al.) стала де-факто базой для эффективного инференса на edge [R6].
2. **Knowledge Distillation** (Hinton et al.) дает путь переносить качество teacher в tiny student [R7].
3. Для TinyML low-precision стратегии показывают нелинейные эффекты: не всегда «меньше бит = лучше», важен баланс между слоями и задачей [R8].
4. На keyword spotting в MCU-сценарии pruning+quantization дают практический компромисс по latency/accuracy [R9].

Вывод для диплома:

1. Научную новизну реально строить вокруг **совместной** оптимизации accuracy + latency + communication cost.
2. Просто «сжали до INT8» недостаточно для сильной защиты; нужна новая целевая функция или routing policy.

3.3 3.3 Распределенный и split инференс

Классическая мотивация: перенести часть вычислений между устройством и edge/сервером для выигрыша в энергии и задержке.

1. **Edgent / co-inference** показывает логику dynamic partitioning + early-exit [R10].
2. **NoNN** (Network of Neural Networks) близко к твоей исходной идее: teacher-модель декомпозируется в набор subnetworks с учетом памяти и коммуникации [R11].
3. Новые работы по split TinyML на реально слабых узлах (в т.ч. ESP32-S3 стенд) подтверждают, что коммуникация часто становится лимитирующим фактором [R12].

Критический вывод:

1. Split по внутренним слоям возможен, но на MCU-сетях часто упирается в объем промежуточных тензоров и jitter канала.
2. Для ESP32-практики обычно выгоднее **decision-level** или **logit-level fusion**, а не «честный layer-split» между микроконтроллерами.

3.4 3.4 Бенчмарки и reproducibility

MLPerf Tiny важен как референс фрейм:

1. Дает стандартные задачи и правила сравнения для tiny inference [R13].
2. Подтверждает, что без нормальной методологии измерений «ускорение» часто нерепродуцируемо.

Вывод:

1. Для диплома нужны формальные протоколы измерений, multiple runs, доверительные интервалы и сравнение с baseline по одним и тем же данным.

3.5 3.5 Аннотированный обзор ключевых статей (что именно брать в дипломный текст)

Ниже не просто список, а «зачем эта статья тебе нужна».

3.5.1 [R2] MCUNet

Что доказывает:

1. Для MCU выигрывает не только сжатие модели, но и совместный дизайн model + runtime.

Как использовать в дипломе:

1. В обзорной главе как аргумент, что pure-ML оптимизация без системного уровня недостаточна.
2. В обосновании собственного communication-aware подхода как следующего шага после memory-aware.

Слабое место:

1. Фокус в первую очередь на single-device inference, а не на мульти-узловой кооперации.

3.5.2 [R3] MCUNetV2

Что доказывает:

1. Ранние слои CNN часто создают memory peak; patch-based inference снижает этот bottleneck.

Как использовать:

1. В разделе «архитектурные трюки для ограниченной памяти».

Слабое место:

1. Может добавить накладные расходы и усложнить deployment pipeline.

3.5.3 [R11] NoNN

Что доказывает:

1. Можно системно декомпозировать teacher-модель в сеть subnetworks с учетом памяти и трафика.

Как использовать:

1. Как прямой теоретический предшественник твоей идеи «блоков на разных MCU».

Слабое место:

1. Практическая реализуемость на конкретном ESP32-стеке требует аккуратной адаптации.

3.5.4 [R10] Edgent / co-inference

Что доказывает:

1. Dynamic partitioning и early-exit дают управляемый trade-off точность/задержка.

Как использовать:

1. В построении adaptive routing policy.

Слабое место:

1. Device-edge сценарий не равен MCU-to-MCU mesh; нужно переосмыслить под ESP-NOW.

3.5.5 [R12] Split-Learning TinyML on ESP32-S3

Что доказывает:

1. Split подход на ультраограниченных устройствах возможен и измерим.

Как использовать:

1. Как современный evidence, что вопрос актуален именно для ESP32-класса.

Слабое место:

1. Важно перепроверять применимость чисел к твоему сетапу; abstract-level цифры не заменяют собственный benchmark.

3.5.6 [R6] Integer-only quantization

Что доказывает:

1. Практически применимый способ перейти к integer вычислениям без катастрофической потери качества.

Как использовать:

1. Как базу твоего этапа компрессии перед deployment.

Слабое место:

1. Не решает автоматически проблемы распределенной коммуникации.

3.5.7 [R7] Distillation

Что доказывает:

1. Student может перенять поведение сильного teacher.

Как использовать:

1. Для обучения маленьких узловых моделей.
2. Для твоего novelty-пункта communication-aware distillation.

Слабое место:

1. Требуется аккуратной настройки температуры, весов loss и контроля overfitting.

3.5.8 [R9] KWS pruning/quantization on low-power MCU

Что доказывает:

1. KWS — реальная tiny-задача с измеримыми trade-off.

Как использовать:

1. Для обоснования выбора предметной области диплома.

Слабое место:

1. Single-device фокус, поэтому кооперативный эффект нужно доказывать самостоятельно.

3.6 3.6 Литературные пробелы (где есть место для твоей новизны)

По текущему корпусу источников, есть явные «дыры»:

1. Мало работ, где одновременно оптимизируются точность, latency, трафик и устойчивость к packet loss для MCU-to-MCU.
2. Мало reproducible тестов для бытовых акустических сценариев с несколькими ESP-узлами.
3. Мало практических adaptive-policy работ, где контроль принимается локально на микроконтроллерах, а не на мощном edge сервере.

Именно здесь твой диплом может быть сильным.

4 4. ESP32/ESP32-S3 для ML: тех-обзор «для чайника, но без упрощенчества»

4.1 4.1 Что важно понять сразу

Для TinyML на ESP32 важны не только «MHz и RAM», а 5 вещей:

1. Где хранятся веса и активации (internal RAM vs PSRAM vs flash).
2. Что делает cache и когда он тебя подводит.
3. Как RTOS планирует задачи на двух ядрах.
4. Что делает Wi-Fi стек с latency/jitter.
5. Что происходит при power-save режимах.

4.2 4.2 ESP32 семейство (практический взгляд)

4.2.1 ESP32 (классический)

1. Двухъядерный Xtensa, до 240 MHz.
2. Ограниченная внутренняя память, хватает для небольших INT8 моделей, но без запаса [R14].

4.2.2 ESP32-S3 (предпочтительный для диплома)

1. Два ядра Xtensa LX7 до 240 MHz.
2. Векторные инструкции, что особенно важно для NN kernels [R15].
3. Лучше поддерживается для современных TinyML сценариев в стеке Espressif (esp-nn, ESP-DL) [R16][R17].

4.2.3 ESP32-C3 (один core, RISC-V)

1. Одно ядро до 160 MHz, меньше вычислительного бюджета.
2. Подходит для «легких» узлов, но для кооперативного аудиоинференса обычно слабее S3 [R18].

4.3 4.3 Память: внутренняя RAM vs PSRAM

Критично:

1. Внутренняя RAM быстрая, но ее мало.
2. PSRAM увеличивает вместимость, но имеет ограничения по пропускной способности и DMA-сценариям [R19].
3. Большие последовательные обращения к внешней памяти могут выбивать cache и ухудшать latency [R19].

Практический вывод:

1. Критичные буферы инференса и real-time очереди держать во внутренней памяти.
2. В PSRAM складывать менее горячие данные (часть буферов, логи, вторичные структуры).
3. Не рассчитывать, что «добавил PSRAM => все стало быстро».

4.4 4.4 RTOS и многозадачность

IDF FreeRTOS на двухъядерных ESP — это SMP-вариант со своей спецификой [R20]:

1. Задачи можно pin-ить к core.
2. Wi-Fi/Bluetooth стек обычно живет на Core 0.
3. Неправильное распределение задач и приоритетов = jitter и потеря дедлайнов.

Практика:

1. Inference task часто удобнее pin-ить на Core 1.
2. Сетевые callback-и должны быть короткими; тяжелая обработка через очередь в low-priority worker.

4.5 4.5 Сеть и задержки: ESP-NOW/Wi-Fi

По ESP-IDF:

1. ESP-NOW — connectionless Wi-Fi протокол, default rate 1 Mbps [R21].
2. Пакетная полезная нагрузка: v1 до 250 B, v2 до 1470 B [R21].
3. Callback-и ESP-NOW работают из high-priority Wi-Fi task; длинные операции там запрещены [R21].

4. Для power-save возможны значимые задержки на уровне DTIM/listen interval [R22].

Практика:

1. Для low-latency кооперации передавать короткие сообщения и минимизировать retransmit.
2. Если нужен стабильный RTT, избегать агрессивных power-save режимов.
3. Закладывать не только среднюю, но и p95/p99 задержку.

Ниже схема datapath из документации ESP-IDF полезна как инженерная опора: она показывает, где в стеке реально появляются буферы и очереди, которые затем превращаются в джиттер end-to-end задержки. Для диплома это важно, потому что модель может быть быстрой, а система медленной.

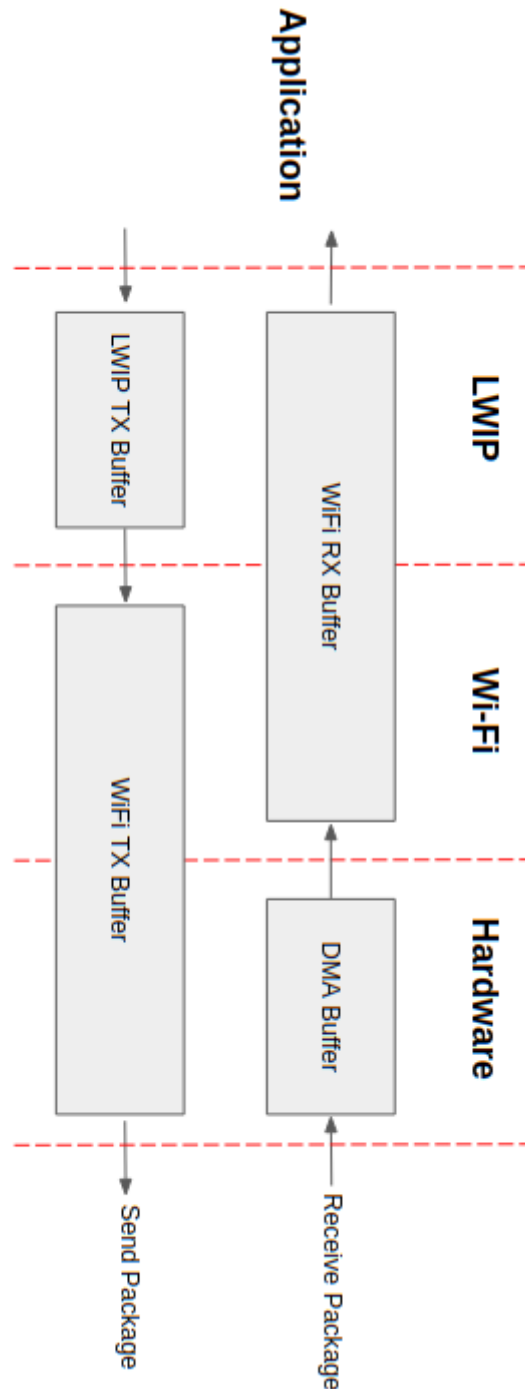


Рис. 2: ESP Wi-Fi datapath из документации ESP-IDF (официальный графический материал Espressif). Источник: [R22].

4.6 Почему задержки в дипломе критичны

Для распределенного инференса полный путь:

$$T_{e2e} = T_{sense} + T_{feat} + T_{infer} + T_{queue} + T_{tx} + T_{fusion} + T_{act}$$

Именно T_{tx} и T_{queue} в реальных ESP-сетях чаще всего «ломают» красивые идеи из чистого ML.

4.7 Мини-курс по микроконтроллерам для ML-специалиста

4.7.1 Что такое MCU на практике

Микроконтроллер — это не «маленький сервер».

Ключевые отличия:

1. Нет большого объема RAM и swap.
2. Нет полноценной ОС с богатым планировщиком и memory isolation.
3. Любая лишняя копия буфера может стоить дедлайна.
4. Любая блокировка в высокоприоритетном callback может положить real-time путь.

4.7.2 Как устроен жизненный цикл программы на ESP32

1. Boot ROM -> загрузчик -> инициализация памяти/тактов -> запуск `app_main`.
2. Дальше приложение обычно раскладывается на RTOS-задачи.
3. Параллельно работает сетевой стек и системные службы.

Для ML важно:

1. Не блокировать системные задачи.
2. Считать инференс регулярной RT-задачей с предсказуемым budget.

4.7.3 Что такое «ядра и пиннинг» в практическом смысле

На двухъядерных ESP (ESP32/ESP32-S3):

1. Можно отделить radio stack от пользовательского ML-пути.
2. Обычно часть протокольной нагрузки сидит на Core 0.
3. Inference и DSP-часть лучше контролируемо размещать на Core 1.

Неправильное решение:

1. «Пусть scheduler сам разберется». На маленьких системах это часто приводит к jitter.

4.7.4 Что такое latency budget

Для каждого 100 ms окна аудио можно задать budget, например:

1. 20 ms на feature extraction.
2. 25 ms на инференс.
3. 10 ms на отправку/прием.
4. 10 ms на fusion и принятие решения.
5. Остальное — запас на jitter.

Если budget нарушается систематически:

1. очередь растёт,
2. окно опаздывает,
3. итоговая ассурасу в реальном времени падает даже при хорошей offline метрике.

4.7.5 Почему offline accuracy может обмануть

В ноутбуке на GPU модель дает условно 96% ассурасу. На MCU в системе могут появиться:

1. пропуски окон,
2. задержанные решения,
3. десинхронизация между узлами,
4. packet loss.

Итог:

1. online-качество может быть заметно хуже offline.
2. Поэтому диплом должен мерить именно end-to-end систему.

4.7.6 Что такое «безопасный» путь к работающему прототипу

1. Сначала полностью локальный single-node pipeline.
2. Затем добавить сеть и serialization без изменения модели.
3. Затем добавить fusion.
4. И только потом adaptive routing и сложные оптимизации.

Это спасает от ситуации, где сразу меняется все и невозможно понять причину деградации.

4.8 4.8 Практические ограничения ESP-NOW именно для ML-пакетов

По официальной документации [R21]:

1. v2 поддерживает payload до 1470 байт, v1 — до 250 байт.
2. Есть ограничения по числу peers.
3. При высокой частоте отправки callback-и могут приходить не в ожидаемом порядке.
4. Отправка должна строиться с учетом подтверждений и таймаутов.

Следствия для протокола диплома:

1. Делать короткие idempotent-пакеты.
2. Обязательно sequence number.
3. Держать совместимость с потерями и дубликатами.

4.9 4.9 Power-save и «скрытые» задержки

Официальные Wi-Fi power-save режимы могут добавить задержку на прием, зависящую от DTIM/listen interval [R22].

Это особенно критично, если ты оцениваешь p95/p99 latency.

Практический вывод:

1. На этапах исследовательского benchmarking лучше сначала отключать агрессивный power-save.
2. Потом отдельно делать серию экспериментов «accuracy/latency vs energy».

5 5. Твоя исходная идея («блоки сети на разных MCU») и ее критический разбор

5.1 5.1 В чем сильная сторона идеи

Идея концептуально сильная:

1. Есть естественная «распределенность».
2. Есть научный простор (поиск лучшего разбиения, маршрутизации, компрессии).
3. Это точно попадает в тему диплома.

5.2 5.2 Главный риск: стоимость передачи активаций

Если делить сеть по слоям между микроконтроллерами, нужно передавать промежуточные тензоры.

Пример оценки:

1. Пусть активация после раннего блока: $49 \times 10 \times 16 = 7840$ элементов.
2. Даже в INT8 это 7.84 KB.
3. На 1 Mbps «чистое» время передачи уже около $7.84 * 8 = 62.7$ ms, без учета overhead и повторов.

Итог:

1. На практике получаешь десятки миллисекунд только на передачу одного промежуточного тензора.
2. Это часто хуже, чем доинференсить локально и отправить короткий вектор логов.

График ниже иллюстрирует ту же проблему в общем виде: задержка передачи растет пропорционально размеру активаций, и даже при умеренном увеличении payload граница real-time быстро теряется.

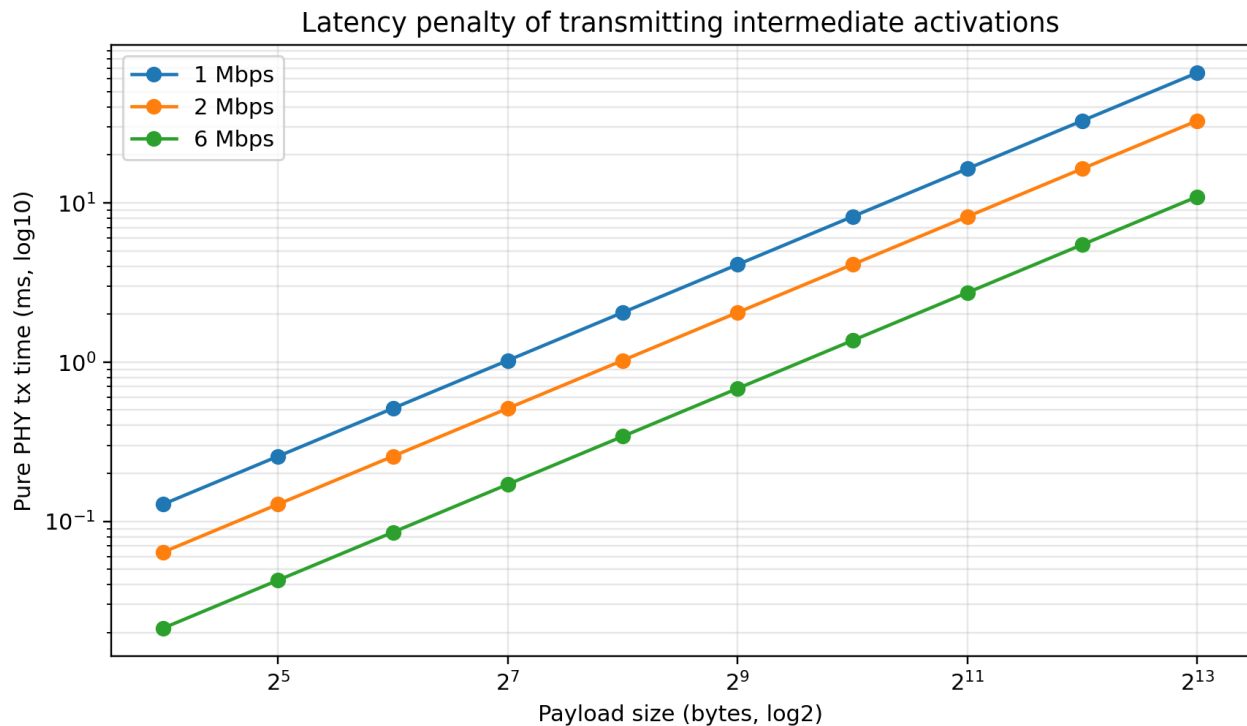


Рис. 3: Зависимость «чистого» времени передачи от размера payload при разных эффективных скоростях канала. Иллюстрация построена по расчетной модели PHY time.

5.3 5.3 Когда блочная идея все же жизнеспособна

1. Если разбиение очень позднее (маленькие тензоры).
2. Если используется высокий и стабильный throughput канал (не типичный ESP-NOW режим).
3. Если есть сильная причина (например, очень тяжелый tail сети).

Вывод:

1. Блочная идея должна быть *модифицирована* в сторону communication-aware подхода, иначе риск провала по latency очень высок.

Практическую границу применимости удобно видеть в фазовом поле «размер активации — скорость канала». Эта карта не заменяет измерения на железе, но хорошо показывает, почему многие наивные точки разреза оказываются технически плохими.

6 5.4 Как спасти исходную «блочную» идею и сделать ее научно сильной

Сама идея «разные блоки сети на разных микроконтроллерах» не только жизнеспособна, но и концептуально правильна для темы диплома. Проблема появляется тогда, когда блочная архитектура трактуется слишком буквально: как статическая последовательность слоев с передачей больших промежуточных карт признаков между узлами. В таком варианте решение обычно упирается в радиоканал быстрее, чем в вычисления. Но это не означает, что от идеи нужно отказаться; это означает, что ее надо переформулировать как задачу совместного проектирования блоков, представлений и маршрутов.

Ниже предложена версия этой концепции в форме **Communication-Constrained Neural Block Graph (CC-NBG)**. Вместо «слой A на узле 1, слой B на узле 2» вводится граф блоков с явными интерфейсами связи. Каждый интерфейс содержит не сырую активацию, а компактный токен фиксированного бюджета, полученный через обучаемый bottleneck-адаптер. Если наивный split передает килобайты, то CC-NBG заставляет систему передавать десятки байт в контролируемом формате.

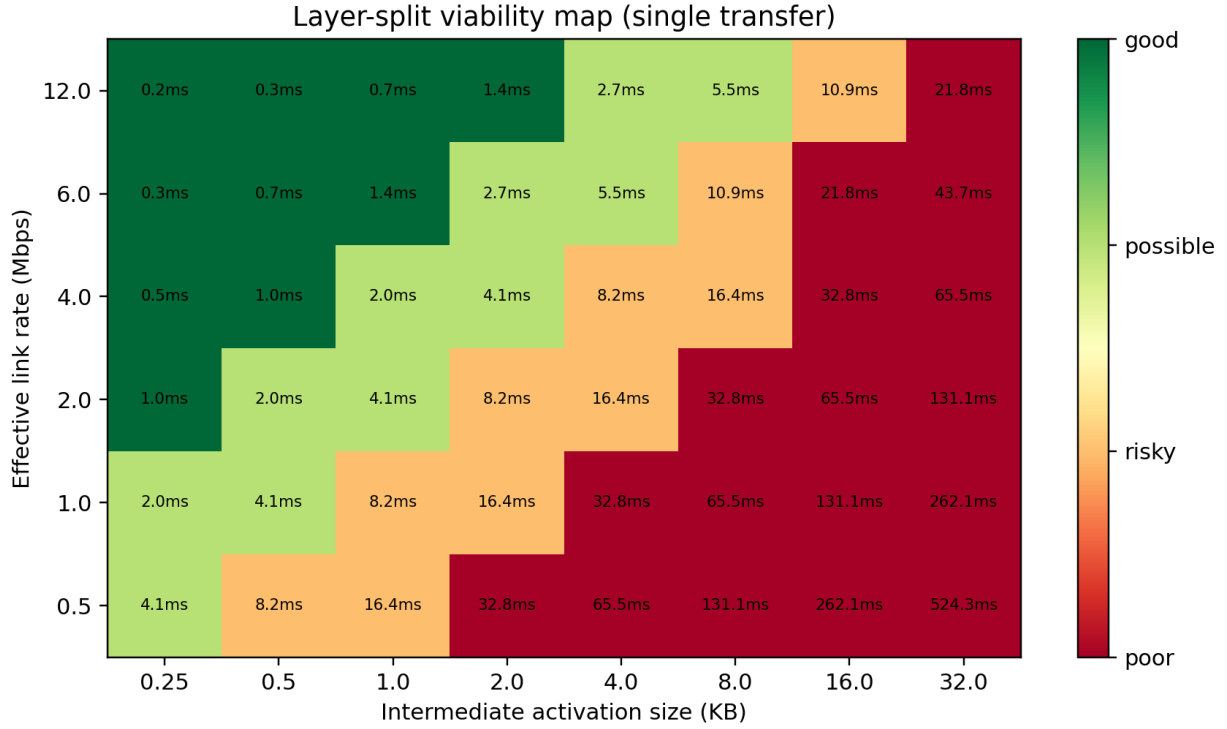


Рис. 4: Карта жизнеспособности layer-split в зависимости от размера промежуточного тензора и скорости канала. Значения в клетках — оценка времени одной передачи.

Формально можно считать, что у нас есть ориентированный граф блоков $G = (V, E)$, где вершины — вычислительные модули B_v , а ребра — каналы передачи токенов между ними. Каждой вершине назначается физический узел $m(v) \in \{1, \dots, N\}$. Для каждого ребра $e = (u, v)$ есть адаптер A_e , который кодирует промежуточное представление блока u в токен t_e длиной d_e бит. Тогда суммарная коммуникационная стоимость для входного потока с частотами активации ребер f_e выражается как

$$C = \sum_{e \in E} f_e \cdot d_e.$$

Задержка же складывается из вычислений, ожиданий и передачи:

$$L = \sum_{v \in V} t_v^{comp} + \sum_{e \in E} \left(t_e^{queue} + \frac{d_e}{r_e} \right),$$

где r_e — эффективная скорость канала по ребру, а не паспортная скорость PHY.

Тогда научная постановка перестает быть «как запустить сеть на нескольких ESP32» и становится многокритериальной задачей:

$$\min_{\theta, A, m(\cdot)} \mathcal{L}_{task} + \lambda_C C + \lambda_L L + \lambda_R \mathcal{L}_{robust},$$

при ограничениях на память каждого узла и на допустимую tail-задержку. В таком виде у тебя появляется и инженерная, и математическая новизна: ты оптимизируешь не только веса сети, но и саму структуру распределенного исполнения.

Эта переформулировка хорошо согласуется с литературой. NoNN показывает, что учет памяти и трафика при декомпозиции teacher-модели — не частный трюк, а отдельная исследовательская ось [R11]. BottleNet и BottleFit показывают, что обучаемые bottleneck-представления могут радикально уменьшать коммуникационную цену split-инференса [R34][R35]. DEFER и работы по edge-кластерам добавляют аспект системного размещения блоков на нескольких устройствах [R38][R39]. Scission подчеркивает, что «лучшее разбиение» сильно контекстно и зависит

от нагрузки и ресурса в моменте [R37]. deepFogGuard добавляет, что без отказоустойчивых связей распределенная DNN хрупка [R36]. В сумме это дает тебе крепкую теоретическую опору именно для исходной блочной идеи.

Главный практический поворот, который и делает концепцию рабочей на ESP32, — это отказ от «жесткой цепочки» в пользу нескольких допустимых маршрутов инференса. Иначе говоря, граф блоков не обязан проходиться всегда одинаково. В простых условиях используется короткий путь с малым количеством межузловых передач; в сложных условиях (низкая уверенность, конфликт блоков, деградация канала) включается альтернативный путь с дополнительным блоком уточнения. Так получается гибридный static+dynamic дизайн, в котором блоки и их размещение фиксируются оффлайн, а маршрут выбирается онлайн.

Эта архитектура принципиально **не привязана к акустике**. Акустический кейс удобен как дешевый и наблюдаемый стенд, но тот же подход переносится на вибродиагностику, IMU/HAR, многосенсорный мониторинг оборудования, где каждый узел обрабатывает локальный поток и передает компактные токены для кооперативного решения. Поэтому ты можешь защищать не «систему распознавания слова», а фундаментальный класс решений: распределенный нейрограф на микроконтроллерах с ограничением коммуникации и задержки.

Если формулировать итоговый исследовательский вклад в одной фразе, он звучит так: **разработан и экспериментально подтвержден метод построения распределенной нейросети на ESP32-кластере, в котором межблочная передача представлений ограничена и оптимизируется совместно с точностью и latency, а выполнение остается устойчивым при вариациях канала и выпадении узлов**. Это уже не частная прикладная склейка, а полноценная научная архитектурная идея.

7 5.5 Универсальный план эксперимента для CC-NBG (не только под аудио)

Чтобы показать, что подход действительно фундаментальный, а не узко-доменный, лучше строить эксперимент в двух слоях. Первый слой — системный, одинаковый для любой предметной области: профилирование блоков, выбор точек разреза, обучение bottleneck-адаптеров, размещение блоков по узлам, измерение latency/energy/traffic при разных сценариях канала. Второй слой — прикладной, где меняется только природа сигнала и метрика задачи.

Практически это можно оформить как две демонстрационные задачи. Первая — акустическая (KWS или event detection), поскольку она доступна по данным и легко демонстрируется вживую. Вторая — неакустическая, например IMU-based activity recognition или виброаномалии, где сенсорный поток и признаки другие, но архитектура распределенного нейрографа остается той же. Если на обеих задачах метод дает похожий системный эффект по трафику и tail-задержке, это очень сильный аргумент в пользу фундаментальности подхода.

8 6. Кандидаты дипломных концептов (с критикой)

8.1 6.1 Кандидат А (рекомендуемый): CC-NBG как базовая архитектура, KWS как первый демонстратор

В этой формулировке основной объект исследования — не «акустическая система», а универсальный распределенный нейрограф на кластере ESP32. Акустическая задача используется только как первый демонстратор, потому что она дешева в постановке и на ней удобно быстро получить воспроизводимые baseline-результаты. После этого тот же каркас переносится на вторую, неакустическую задачу (например, IMU/HAR или вибродиагностику), и именно переносимость архитектуры становится аргументом фундаментальности.

Сильная сторона этого варианта в том, что он одновременно дает инженерную реалистичность и научный вклад. Инженерная часть опирается на конкретный стек ESP-IDF/ESP-NOW и измеримые ограничения канала. Научная часть формализуется через оптимизацию разбиения блоков, размера передаваемых токенов и adaptive routing policy под latency/traffic budget. Слабое место подхода — более высокий объем работы по сравнению с «одной прикладной задачей», потому что нужно аккуратно поставить как минимум два домена валидации. Но именно это и отделяет фундаментальный диплом от локального прототипа.

8.2 6.2 Кандидат В: Распределенное обнаружение бытовых аномалий (звук + вибрация + газ)

Суть:

1. Разные сенсорные узлы с разными tiny-моделями.

2. Fusion на координаторе.

Плюсы:

1. Мультимодальность интересна научно.
2. Хороший narrative для «умного дома».

Минусы:

1. Данные по реальным авариям добывать тяжело.
2. Сложнее объективно валидировать «лучше».

8.3 6.3 Кандидат С: Чистый layer-split DNN между ESP32

Суть:

1. Разные блоки одной сети крутятся на разных узлах.

Плюсы:

1. Красивая теоретически архитектура.

Минусы:

1. Очень высокий риск провала по latency из-за передачи активаций.
2. Большой engineering overhead и сложность отладки.

Вердикт: скорее как контрольный baseline/эксперимент, но не как основной дипломный продукт.

8.4 6.4 Кандидат D: Федеративная персонализация tiny-моделей на ESP32

Плюсы:

1. Научно модно.

Минусы:

1. Слишком много рисков: обучение на MCU, память, коммуникация, устойчивость.
2. Для диплома «один год» может быть перегруз.

Вердикт: как направление future work.

8.5 6.5 Сравнительная матрица (формализованный выбор)

Шкала: 1 (плохо) .. 5 (сильно).

Критерий	A: CC-NBG (универсальная архитектура + KWS demo)	B: Бытовые аномалии	C: Чистый layer-split	D: Federated on-device
Соответствие теме диплома	5	5	5	4
Реализуемость за дипломный срок	4	3	2	2
Стоимость стенда	4	3	4	4
Научная новизна	5	4	4	5
Риск провала по latency	2	3	5	4
Доступность данных	4	2	4	3
Понятность демонстрации комиссии	5	3	3	2

Критерий	A: CC-NBG (универсальная архитектура + KWS demo)	B: Бытовые аномалии	C: Чистый layer-split	D: Federated on-device
Итоговый приоритет	29/35	23/35	27/35 (но высокий риск)	24/35

Комментарий:

1. Кандидат C теоретически привлекателен, но без bottleneck-адаптеров и динамической маршрутизации обычно проигрывает по tail-latency.
2. Кандидат A выигрывает тем, что формулирует фундаментальную архитектурную задачу и при этом остается выполнимым в дипломный срок.

9 7. Лучшая идея: детальная проработка

9.1 7.1 Постановка задачи

Цель:

Построить и экспериментально проверить универсальную архитектуру CC-NBG на ESP32-S3-кластере, где распределенные нейросетевые блоки совместно решают задачу инференса при ограничениях по памяти, трафику и задержке. Демонстрация проводится как минимум на двух предметных сценариях (один может быть акустическим, второй — неакустическим), чтобы показать переносимость архитектурного подхода.

Объект сравнения:

1. Baseline-1: одиночный узел с локальной tiny-моделью без кооперации.
2. Baseline-2: статический layer-split без bottleneck-адаптеров.
3. Baseline-3: централизованный сбор признаков/токенов без adaptive routing.
4. Baseline-4 (опционально): cloud-assisted инференс.

9.2 7.2 Архитектура системы

9.2.1 Узлы (Node-i)

Каждый узел:

1. Читает локальный сенсорный поток (например, I2S-микрофон или IMU/вибродатчик).
2. Строит признаки, релевантные выбранной предметной задаче.
3. Запускает локальный tiny-блок инференса (модель или фрагмент модели).
4. Оценивает confidence и качество входного канала.
5. Отправляет компактный пакет в координатор.

9.2.2 Координатор

1. Принимает пакеты от узлов (ESP-NOW).
2. Выполняет weighted fusion логитов.
3. При низкой уверенности запрашивает second-stage у выбранного узла.
4. Выдает финальный детект.

9.2.3 Почему это communication-efficient

Передается пакет порядка десятков байт, а не килобайты сырых признаков/активаций.

9.3 7.3 Формальная математическая модель

Пусть есть N узлов.

Локальная модель узла i :

$$z_i = f_i(x_i; \theta_i), \quad p_i = \text{softmax}(z_i)$$

Локальная неопределенность:

$$u_i = H(p_i) = - \sum_c p_i(c) \log p_i(c)$$

Надежность узла:

$$w_i = \phi(\text{SNR}_i, \text{RSSI}_i, 1 - u_i)$$

Слияние (пример: взвешенный product/logit fusion):

$$\hat{z} = \sum_{i=1}^N w_i z_i, \quad \hat{p} = \text{softmax}(\hat{z})$$

Решение:

$$\hat{y} = \arg \max_c \hat{p}(c)$$

Communication-aware цель обучения:

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda_1 \mathbb{E}[B] + \lambda_2 \mathbb{E}[T_{e2e}] + \lambda_3 \mathcal{L}_{KD}$$

где:

1. B — число переданных бит.
2. T_{e2e} — end-to-end latency.
3. $\mathcal{L}_{KD}$ — дистилляция от teacher-модели.

Это и есть научная «склейка» ML + системной оптимизации.

9.4 7.4 Routing policy (ядро новизны)

Пример политики:

1. Если $u_i < \tau_1$, узел может отдать локальное решение (экономия трафика).
2. Если $\tau_1 \leq u_i < \tau_2$, узел инициирует кооперативный fusion.
3. Если $u_i \geq \tau_2$, активируется second-stage (более тяжелый классификатор) только для сложных случаев.

Преимущество:

1. «Легкие» примеры проходят быстро и дешево.
2. «Тяжелые» получают дополнительную обработку.

9.5 7.5 Почему это лучше для ESP32 именно с учетом задержек

9.5.1 Сравнение коммуникационной нагрузки

Вариант 1: передать промежуточную активацию 7.84 KB

Вариант 2: передать 12 логитов + метаданные, например 32–64 B

При 1 Mbps:

1. 7.84 KB -> десятки миллисекунд только на физическую передачу.
2. 64 B -> доли миллисекунды на payload (плюс overhead).

Именно поэтому для ESP-NOW обычно выгоднее decision/logit-level кооперация.

9.5.2 Реальный jitter

Даже при хорошей средней задержке система может «сыпаться» по p95/p99 из-за:

1. очередей Wi-Fi task,
2. коллизий в эфире,
3. фоновых задач,
4. power-save циклов [R22].

Поэтому метрика «среднее время» недостаточна.

9.6 7.6 Предлагаемая двухуровневая архитектура (детально)

Первый уровень — это быстрый always-on маршрут, который должен укладываться в жесткий latency budget и работать даже при неблагоприятном канале. На этом уровне каждый узел выполняет минимально достаточный локальный инференс, формирует компактный токен (logits/latent + confidence + quality) и отправляет его координатору. На практике конкретный front-end зависит от домена: для аудио это могут быть log-mel признаки, для IMU — спектрально-временные признаки, для вибродиагностики — энергетические и частотные дескрипторы.

Второй уровень включается не всегда, а только по условию неопределенности или конфликтности решения. Если энтропия высокая, если узлы дают взаимно противоречивые гипотезы или если канал деградировал так, что решение становится ненадежным, система переключается на уточняющий маршрут. Уточнение может выполняться на координаторе более тяжелым блоком, на «лучшем» узле с высоким quality score или через короткое дополнительное окно наблюдения. Системный смысл двухуровневой схемы в том, что легкие примеры обрабатываются быстро и дешево, а тяжелые получают дополнительный вычислительный ресурс только тогда, когда это реально нужно.

Для диплома здесь появляется удобная исследовательская ось: как выбор порогов τ_1, τ_2 , политики переключения и бюджета токена меняет одновременно ассурасу, latency и энергопотребление. Именно эта совместная зависимость делает тему научной, а не просто инженерной.

9.7 7.7 Как обучать под distributed режим, а не просто «обычный KWS»

Обучающий pipeline лучше строить в несколько последовательных фаз. Сначала обучается teacher-модель на полной задаче и расширенном наборе условий (включая шумы, помехи и вариации домена), чтобы получить устойчивый источник soft targets. Затем строятся узловые student-блоки под MCU-бюджет, и в этот этап уже добавляется distillation вместе с quantization-aware training. Важно, что для distributed-сценария student должен учиться не только классифицировать, но и производить информативные промежуточные представления ограниченного объема.

После этого оффлайн готовится fusion/routing policy: собирается датасет токенов, confidence/quality признаков и истинных меток, на котором подбирается либо компактная fusion-модель, либо строгая rule-based политика. Финальная обязательная фаза — инъекция сетевых сбоев в тренировочную и тестовую петлю: packet dropout, вариативная задержка и выпадение узлов. Без этой фазы результаты на реальном стенде почти всегда оказываются хуже «чистого» оффлайн-эксперимента, и это один из главных источников разрыва между красивой идеей и рабочей системой.

9.8 7.8 Анти-гипотезы, которые нужно честно проверять

Чтобы диплом был научным, надо попытаться опровергнуть собственную идею.

Анти-гипотеза H0-1:

1. Кооперация нескольких ESP не дает значимого выигрыша против одного ESP.

Анти-гипотеза H0-2:

1. Выигрыш в ассурасу съедается ростом latency.

Анти-гипотеза H0-3:

1. В реальных бытовых шумах adaptive routing нестабилен.

Если ты заранее закладываешь такие проверки, защита выглядит сильно.

9.9 7.9 Memory budget: как быстро оценивать влезаемость модели

Приближенная оценка RAM на инференс:

$$M_{total} \approx M_{weights} + M_{activations_peak} + M_{runtime} + M_{buffers}$$

Где:

1. $M_{weights}$: размер квантованных весов (INT8 обычно 1 байт на вес).
2. $M_{activations_peak}$: пиковый размер промежуточных тензоров.
3. $M_{runtime}$: служебные структуры TFLM/ESP-DL.
4. $M_{buffers}$: аудио ring buffers, очереди, сетевые буферы.

Практическое правило:

1. Держать запас не меньше 20–30% внутренней памяти на системные пики.
2. Не планировать «под ноль», иначе стабильность падает.

9.10 7.10 Почему ESP32-S3 предпочтительнее классического ESP32 для этой задачи

1. Векторные инструкции и оптимизации esp-nn дают заметный прирост kernels [R16].
2. Более удобный профиль для современных TinyML пайплайнов с несколькими задачами.
3. В кооперативной системе важна не только «голая» скорость инференса, но и запас по планировщику и буферам.

10 8. Экспериментальный дизайн (чтобы защита была сильной)

10.1 8.1 Аппаратный стенд

Рекомендуемый минимальный стенд:

1. 4x ESP32-S3 DevKit.
2. 4x I2S MEMS микрофон.
3. 1x ESP32-S3 координатор (можно совмещать с одним из узлов, но лучше отдельно).
4. Питание от USB/PowerBank + измеритель тока.

10.2 8.2 Программный стек

1. ESP-IDF как основа.
2. esp-nn для ускоренных kernels [R16].
3. esp-tflite-micro или ESP-DL для runtime [R17][R23].
4. ESP-NOW API из esp-idf [R21].

10.3 8.3 Данные

1. Public: Speech Commands [R24].
2. Шумы: MUSAN/домашние шумы (TV, кухня, пылесос, улица).
3. Собственные записи в 2–3 комнатах с разными позициями источника.

10.4 8.4 Бейзлайны и абляции

Бейзлайны:

1. Single-node KWS.
2. Centralized raw feature sharing.
3. (Опционально) cloud API baseline.

Абляции:

1. Без distillation.
2. Без communication penalty в loss.
3. Без adaptive routing (всегда fusion).
4. Разное число узлов (2/3/4/5).

10.5 8.5 Метрики

Обязательные:

1. Accuracy / F1.
2. FAR (false alarms per hour).
3. FRR (false reject rate).
4. p50/p95/p99 latency.
5. Энергия на 1 событие.
6. Пакетные потери и устойчивость к ним.

10.6 8.6 Статистика

1. 5–10 повторов на сценарий.
2. Bootstrap CI для latency и FAR.
3. McNemar test для сравнения классификаторов на одинаковых примерах.

10.7 8.7 Протокол сценариев (чтобы результаты были валидны)

Минимальный набор сценариев:

1. Тихая комната, близкая речь.
2. Тихая комната, дальняя речь.
3. Фоновый TV шум.
4. Кухонный шум/посудомойка.
5. Несколько говорящих (interference).
6. Частичный packet loss (эмуляция 5/10/20%).

Для каждого сценария:

1. фиксированная длительность сессии,
2. фиксированное число целевых команд,
3. протокол запуска узлов и синхронизации.

10.8 8.8 Как измерять энергию корректно

Если есть измеритель тока:

$$E = \int_0^T V \cdot I(t) dt$$

Практически:

1. измеряешь $I(t)$ с частотой, достаточной для пиков радио и инференса,
2. считаешь энергию на 1 событие и на 1 час работы.

Если нет точного измерителя:

1. использовать относительные сравнения по duty-cycle и среднему току в повторяемых режимах.

10.9 8.9 KPI для итоговой защиты

Рекомендуемые целевые формулировки:

1. Снижение FAR не менее чем на X% при одинаковом FRR.
2. p95 latency не выше заданного порога (например, 200 ms end-to-end).
3. Рост энергопотребления не более Y% относительно single-node baseline.

Важно:

1. X и Y зафиксировать после пилотной серии, а не придумывать в конце.

11 9. Инженерные детали реализации

11.1 9.1 Рекомендованная структура задач FreeRTOS

На каждом узле:

1. `audio_task` (high): сбор кадров, ring buffer.
2. `feature_task` (high/medium): MFCC/log-mel.
3. `infer_task` (medium): TFLM/ESP-DL inference.
4. `radio_tx_task` (medium): отправка пакетов.
5. `control_task` (low): конфиг, heartbeat.

Важно:

1. Сетевые callback-и не перегружать вычислениями.
2. Между callback и инференсом только очереди/lock-free буферы.

11.2 9.2 Формат пакета ESP-NOW

Пример:

1. `node_id` (1B)
2. `seq` (2B)
3. `timestamp_local` (4B)
4. `logits_q7` (12B для 12 классов)
5. `confidence` (1B)
6. `snr_bin` (1B)
7. `crc16` (2B)

Итого: около 24–32B.

11.3 9.3 Синхронизация времени

Для аудиоокна важен относительный тайминг:

1. Периодический beacon от координатора.
2. Коррекция локального сдвига часов.
3. Допуск окна fusion, например ± 20 ms.

11.4 9.4 Обработка потерь пакетов

1. Sequence number + дедупликация.
2. Ограниченный ретрансмит.
3. Если пакет не пришел — fusion по доступным узлам с корректировкой весов.

11.5 9.5 Рекомендуемая структура репозитория

```
project/  
  data/  
  training/  
    teacher/  
    student/  
    distill/  
    quant/  
  firmware/  
    node/  
    coordinator/  
    common_proto/  
  tools/  
    log_parser/  
    latency_analyzer/  
    power_analyzer/  
  experiments/  
    configs/
```

```
results/  
docs/
```

Плюс:

1. отдельный README с командами воспроизведения.
2. фиксированные config файлы для всех серий экспериментов.

11.6 9.6 Минимальный протокол логирования

На каждом узле:

1. seq, timestamp_capture, timestamp_infer_done, confidence, class_id.

На координаторе:

1. timestamp_pkt_rx, fusion_window_id, decision_time, decision, source_nodes.

Зачем:

1. без такого лога невозможно корректно посчитать end-to-end latency и разобрать деградации.

11.7 9.7 Типичные embedded-баги в таких проектах

1. Непредсказуемые malloc/free в hot path.
2. Большие memcpu внутри callback.
3. Лишний printf в high-frequency task.
4. PSRAM-буферы в местах, где нужна низкая latency.
5. Блокирующие ожидания в радиозадачах.

Профилактика:

1. pre-allocated буферы,
2. профилирование по этапам конвейера,
3. watchdog и health metrics.

12 10. Кодовый скелет (концептуально)

```
// Node side pseudo-code (ESP-IDF style)  
void infer_task(void* arg) {  
    while (true) {  
        audio_frame_t frame = pop_audio_frame();  
        feature_t feat = compute_logmel(frame);  
        logits_t z = kws_infer(feat);           // INT8 model  
        uint8_t conf = quantize_confidence(z);  
        uint8_t snr = estimate_snr_bin(frame);  
  
        packet_t pkt = make_packet(node_id, next_seq(), now_us(), z, conf, snr);  
        enqueue_radio_tx(pkt);  
    }  
}  
  
void radio_send_cb(...) {  
    // minimal work only  
    post_send_status_to_queue(...);  
}  
  
// Coordinator side pseudo-code  
void fusion_task(void* arg) {  
    while (true) {  
        window_packets_t W = collect_packets(time_window_ms);  
        fused_logits_t zf = weighted_fusion(W); // weights from SNR/confidence/history  
  
        if (entropy(zf) > TAU2) {
```

```

    request_second_stage(best_node(w));
    zf = refine_with_stage2(zf);
}

decision_t d = argmax(zf);
publish_decision(d);
}
}

```

13 11. Где именно научная новизна (чтобы комиссия не сказала «это просто инженерка»)

13.1 11.1 Novelty 1: Communication-aware distillation

Teacher обучается централизованно, а student-узлы получают loss, штрафующий трафик:

$$\mathcal{L}_{student} = \alpha \cdot CE(y, p_s) + \beta \cdot KL(p_t || p_s) + \gamma \cdot \text{BitsPenalty}$$

Где BitsPenalty зависит от частоты отправки и размера сообщений.

13.2 11.2 Novelty 2: Latency-aware adaptive routing

Политика выбора режима (local-only, fusion, stage2) оптимизирует:

$$\max U = \text{AccuracyGain} - \eta_1 \cdot T_{e2e} - \eta_2 \cdot E$$

Это можно реализовать как:

1. rule-based policy (проще и надежнее для диплома),
2. lightweight contextual bandit (если хочешь усилить исследовательскую часть).

13.3 11.3 Novelty 3: Robust fusion under node/link failures

Обучение и тестирование с искусственным dropout узлов/каналов, чтобы показать устойчивость системы к сетевой деградации.

13.4 11.4 Как формально сформулировать вклад в ВКР

Удобная формула «вкладов»:

1. Предложена архитектура распределенной TinyML-системы на ESP32-S3 с двухуровневым адаптивным инференсом.
2. Разработан communication-aware критерий обучения student-моделей с учетом budget трафика и latency.
3. Предложен и экспериментально оценен механизм устойчивого fusion при частичной потере узлов/пакетов.
4. Получены экспериментальные результаты в бытовых акустических сценариях, показывающие преимущество над baseline-решениями.

Это формулируется четко, проверяемо и без «воды».

14 12. Слабые места проекта (критика заранее)

14.1 12.1 Риск синхронизации

Проблема:

Если окна на узлах не совпадают, fusion теряет смысл.

Митигировать:

1. протокол времени,
2. буферизация с компенсирующим сдвигом,
3. явный timestamp в каждом пакете.

14.2 12.2 Риск переусложнения

Проблема:

Слишком много «фич» может убить сроки.

Митигировать:

1. сначала MVP: 3 узла, однофазный fusion.
2. потом second-stage и adaptive routing.

14.3 12.3 Риск недоказанной пользы

Проблема:

Можно сделать красивую систему без статистически значимого выигрыша.

Митигировать:

1. заранее зафиксировать гипотезы и метрики.
2. собрать стресс-сценарии, где одиночный узел слаб (шум, дальняя речь, реверберация).

14.4 12.4 Риск «сетевое узкое место»

Проблема:

При большом трафике ESP-NOW деградирует.

Митигировать:

1. ultra-compact payload,
2. event-driven отправка,
3. контроль частоты и duty cycle,
4. fallback на local-only в перегрузке.

15 13. Почему этот диплом реально может быть «лучше существующих решений»

Формулируем честно, без маркетинга:

1. Не «лучше всех вообще», а **лучше одиночного MCU baseline и простых централизованных схем в конкретном классе сценариев.**
2. Сценарий: шумная домашняя среда и распределенные источники звука.
3. Критерий: меньше ложных тревог и пропусков при заданном или меньшем p95 latency.

Это инженерно доказуемо и академически корректно.

15.1 13.1 Пример ожидаемой таблицы результатов (шаблон)

Метод	F1	FAR/час	FRR	p95 latency (ms)	Energy/event (mJ)
Single-node ESP32-S3	...	...	...	...	...
Centralized raw features	...	...	...	...	...
Proposed distributed adaptive	...	...	...	...	...

Комментарий:

1. В таблицу заносить среднее и 95% CI.
2. Отдельно показывать breakdown по сценариям шума.

15.2 13.2 Что считать «успехом диплома»

Минимальный успех:

1. Система стабильно работает end-to-end.
2. Есть статистически значимый выигрыш по хотя бы двум метрикам.

Сильный успех:

1. Выигрыш одновременно по robustness и по p95 latency в ключевых сценариях.
2. Показан устойчивый режим при packet loss без драматического падения качества.

16 14. Дорожная карта диплома (пример на 16 недель)

1. Недели 1–2: финализация постановки, метрик, протокола эксперимента.
2. Недели 3–4: single-node baseline, воспроизводимый pipeline обучения.
3. Недели 5–6: подъем ESP-NOW кооперации 2–3 узлов.
4. Недели 7–8: fusion v1 + измерение latency/energy.
5. Недели 9–10: distillation + quantization + абляции.
6. Недели 11–12: adaptive routing + fault-tolerance тесты.
7. Недели 13–14: масштабирование до 4–5 узлов, полный benchmark.
8. Недели 15–16: оформление текста, графиков, репозитория, демо.

Proposed methodology for communication-aware distributed MCU inference

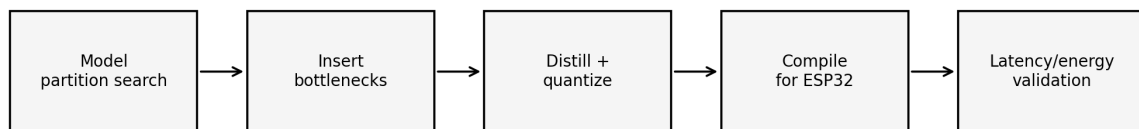


Рис. 5: Методологический конвейер исследования: от поиска разрезов и bottleneck-модулей до валидации latency/energy на железе.

17 15. Практические рекомендации «что делать завтра»

1. Выбрать фиксированный MVP: 3 узла ESP32-S3, задача KWS.
2. Сразу заложить сбор метрик latency (p50/p95/p99) и FAR/FRR.
3. Сделать два baseline-а до «наворотов».
4. Только после этого добавлять adaptive routing и communication-aware loss.

18 16. Заключение

Твоя исходная идея сильная, но в чистом виде (layer blocks across MCU) для ESP32 часто бьется о задержки и трафик.

Лучший путь для диплома под тему «Распределенная сеть нейронных сетей на микроконтроллерах»:

1. распределенная сеть **узлов-экспертов** (каждый со своей tiny-моделью),
2. communication-efficient кооперация через короткие сообщения,
3. научная новизна в совместной оптимизации точности, задержки и трафика.

Это одновременно:

1. реализуемо на доступном железе,

2. проверяемо на бытовых данных/сценариях,
 3. достаточно «научно», чтобы хорошо защищаться.
-

19 Приложение А. Быстрый глоссарий (ML -> embedded)

1. **Internal RAM:** быстрая память на кристалле, дефицитная.
 2. **PSRAM:** внешняя память, больше объем, но хуже предсказуемость доступа.
 3. **Flash:** где хранится прошивка и веса в неоперативной памяти.
 4. **IRAM/DRAM:** области памяти под код/данные для быстрого выполнения.
 5. **RTOS task:** «поток» в микроконтроллерной ОС.
 6. **Pinning task to core:** привязка задачи к конкретному ядру.
 7. **p95 latency:** 95-й перцентиль задержки (почти worst-case, но не экстремум).
-

20 Приложение В. Источники и ссылки

20.1 B.1 TinyML, distributed inference, optimization

[R1] CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs
<https://arxiv.org/abs/1801.06601>

[R2] MCUNet: Tiny Deep Learning on IoT Devices
<https://arxiv.org/abs/2007.10319>

[R3] MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning
<https://arxiv.org/abs/2110.15352>

[R4] On-Device Training Under 256KB Memory
<https://arxiv.org/abs/2206.15472>

[R5] MEMA Runtime Framework: Minimizing External Memory Accesses for TinyML on Microcontrollers
<https://arxiv.org/abs/2304.05544>

[R6] Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference
<https://arxiv.org/abs/1712.05877>

[R7] Distilling the Knowledge in a Neural Network
<https://arxiv.org/abs/1503.02531>

[R8] An Empirical Study of Low Precision Quantization for TinyML
<https://arxiv.org/abs/2203.05492>

[R9] Keyword Spotting System and Evaluation of Pruning and Quantization Methods on Low-power Edge Microcontrollers
<https://arxiv.org/abs/2208.02765>

[R10] Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy
<https://arxiv.org/abs/1806.07840>

[R11] Memory- and Communication-Aware Model Compression for Distributed Deep Learning Inference on IoT (NoNN)
<https://arxiv.org/abs/1907.11804>

[R12] An Experimental Study of Split-Learning TinyML on Ultra-Low-Power Edge/IoT Nodes
<https://arxiv.org/abs/2507.16594>

[R13] MLPerf Tiny Benchmark
<https://arxiv.org/abs/2106.07597>

[R24] Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition
<https://arxiv.org/abs/1804.03209>

[R30] Widening Access to Applied Machine Learning with TinyML
<https://arxiv.org/abs/2106.04008>

- [R31] Efficient Keyword Spotting by Capturing Long-Range Interactions with Temporal Lambda Networks
<https://arxiv.org/abs/2104.08086>
- [R32] MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications
<https://arxiv.org/abs/1704.04861>
- [R33] Distributed Deep Neural Networks over the Cloud, the Edge and End Devices
<https://arxiv.org/abs/1709.01921>
- [R34] BottleNet: A Deep Learning Architecture for Intelligent Mobile Cloud Computing Services
<https://arxiv.org/abs/1902.01000>
- [R35] BottleFit: Learning Compressed Representations in Deep Neural Networks for Effective and Efficient Split Computing
<https://arxiv.org/abs/2201.02693>
- [R36] Guardians of the Deep Fog: Failure-Resilient DNN Inference from Edge to Cloud
<https://arxiv.org/abs/1909.00995>
- [R37] Scission: Performance-driven and Context-aware Cloud-Edge Distribution of Deep Neural Networks
<https://arxiv.org/abs/2008.03523>
- [R38] DEFER: Distributed Edge Inference for Deep Neural Networks
<https://arxiv.org/abs/2201.06769>
- [R39] Partitioning and Deployment of Deep Neural Networks on Edge Clusters
<https://arxiv.org/abs/2304.11941>
- [R40] Neurosurgeon (ASPLOS 2017, DOI)
<https://doi.org/10.1145/3037697.3037698>
- [R41] SpikeBottleNet: Spike-Driven Feature Compression Architecture for Edge-Cloud Co-Inference
<https://arxiv.org/abs/2410.08673>

20.2 B.2 ESP32 official documentation and tooling

- [R14] ESP32 Datasheet (official)
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [R15] ESP32-S3 Datasheet (official)
https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [R18] ESP32-C3 Datasheet (official)
https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf
- [R16] ESP-NN (optimized NN kernels for Espressif chips)
<https://github.com/espressif/esp-nn>
- [R17] TensorFlow Lite Micro for Espressif (esp-tflite-micro)
<https://github.com/espressif/esp-tflite-micro>
- [R23] ESP-DL documentation / component
<https://docs.espressif.com/projects/esp-dl/en/latest/>
<https://components.espressif.com/components/espressif/esp-dl>
- [R25] ESP-DL quantization tutorial
https://docs.espressif.com/projects/esp-dl/en/latest/tutorials/how_to_quantize_model.html
- [R21] ESP-IDF ESP-NOW reference (official source docs)
https://raw.githubusercontent.com/espressif/esp-idf/master/docs/en/api-reference/network/esp_now.rst
- [R22] ESP-IDF Wi-Fi performance and power save (official source docs)
<https://raw.githubusercontent.com/espressif/esp-idf/master/docs/en/api-guides/wifi-driver/wifi-performance-and-power-save.rst>
- [R20] ESP-IDF FreeRTOS SMP details
https://raw.githubusercontent.com/espressif/esp-idf/master/docs/en/api-reference/system/freertos_idf.rst
- [R19] ESP-IDF external RAM / PSRAM guide
<https://raw.githubusercontent.com/espressif/esp-idf/master/docs/en/api-guides/external-ram.rst>

20.3 B.3 Additional ecosystem references

[R26] MLCommons Tiny benchmark repository
<https://github.com/mlcommons/tiny>

[R27] MLCommons Tiny benchmark results portal
<https://mlcommons.org/benchmarks/inference-tiny/>

[R28] Developer Portal article: ESP-NOW reliability (indoor)
<https://developer.espressif.com/blog/reliability-esp-now/>

[R29] Developer Portal article: Arduino ESP-NOW library overview
<https://developer.espressif.com/blog/2024/08/arduino-esp-now-lib/>

21 Приложение С. Что еще можно добавить в финальную версию диплома

1. Полный .bib и оформление по ГОСТ.
 2. Отдельная глава с угрозами валидности (internal/external validity).
 3. Репозиторий с автоматическим прогоном benchmark.
 4. Демо-видео с живым сравнением baseline vs distributed.
-

22 Приложение D. Расширенная аннотированная библиография

Ниже краткие, но практичные аннотации: что дает источник, куда вставлять в ВКР, и где ограничения.

22.1 D.1 Системные и runtime-работы

22.1.1 [R1] CMSIS-NN

1. **Суть:** оптимизированные NN kernels на MCU.
2. **Тезис для диплома:** низкоуровневые kernels радикально влияют на latency/energy.
3. **Куда вставлять:** глава обзора и раздел «связанные работы».
4. **Ограничение:** не покрывает distributed-cooperation уровень.

22.1.2 [R2] MCUNet

1. **Суть:** co-design модели и runtime для tiny устройств.
2. **Тезис:** нельзя оптимизировать только архитектуру сети в отрыве от платформы.
3. **Куда вставлять:** обоснование системного подхода.
4. **Ограничение:** упор на single-device.

22.1.3 [R3] MCUNetV2

1. **Суть:** patch inference снижает memory peak.
2. **Тезис:** пиковая память важнее средней.
3. **Куда вставлять:** раздел по memory bottleneck.
4. **Ограничение:** требует аккуратной инженерной реализации.

22.1.4 [R5] МЕМА

1. **Суть:** минимизация внешних memory accesses.
2. **Тезис:** внешняя память часто становится узким местом.
3. **Куда вставлять:** раздел по PSRAM/кэшу/пропускной способности.
4. **Ограничение:** не затрагивает сетевую кооперацию.

22.1.5 [R4] On-device training under 256KB

1. **Суть:** обучение/дообучение на сверхограниченной памяти.
2. **Тезис:** персонализация на устройстве возможна, но очень дорогая по системным ресурсам.
3. **Куда вставлять:** future work или дополнительная исследовательская ветка.
4. **Ограничение:** для базового дипломного трека может быть перегруз.

22.2 D.2 Компрессия, квантование, дистилляция

22.2.1 [R6] Integer-only quantization

1. **Суть:** integer-only inference как практический стандарт edge.
2. **Тезис:** INT8 это не «хак», а фундаментальная инженерная практика.
3. **Куда вставлять:** методика подготовки student-моделей.
4. **Ограничение:** quantization сам по себе не решает распределенную часть.

22.2.2 [R7] Distillation

1. **Суть:** student копирует поведение teacher через soft targets.
2. **Тезис:** можно сохранить качество при меньшем объеме модели.
3. **Куда вставлять:** раздел обучения локальных узлов.
4. **Ограничение:** требует аккуратной настройки и проверок на доменный сдвиг.

22.2.3 [R8] Low precision quantization for TinyML

1. **Суть:** эмпирика по ultra-low precision режимам.
2. **Тезис:** «меньше бит» не всегда выгоднее.
3. **Куда вставлять:** обоснование выбора именно INT8/смешанной точности.
4. **Ограничение:** переносимость между задачами ограничена.

22.2.4 [R9] KWS pruning + quantization на low-power MCU

1. **Суть:** практический KWS в условиях MCU-ограничений.
2. **Тезис:** KWS отлично подходит как дипломный testbed.
3. **Куда вставлять:** мотивация предметной задачи.
4. **Ограничение:** single-node постановка.

22.3 D.3 Distributed/split inference

22.3.1 [R10] Edgent

1. **Суть:** dynamic model partitioning + early exit.
2. **Тезис:** вычисления можно адаптивно распределять под SLA.
3. **Куда вставлять:** мотивировка adaptive routing.
4. **Ограничение:** device-edge, а не MCU-mesh.

22.3.2 [R11] NoNN

1. **Суть:** memory+communication-aware декомпозиция teacher в сеть subnetworks.
2. **Тезис:** твоя начальная идея имеет сильный научный фундамент.
3. **Куда вставлять:** раздел «прототипы распределенных архитектур».
4. **Ограничение:** нужно адаптировать к реальным ограничителям ESP-NOW.

22.3.3 [R12] Split-Learning TinyML on ultra-low-power nodes

1. **Суть:** практический split-learning тестбед на слабых IoT-устройствах.
2. **Тезис:** split-подход реализуем, но чувствителен к каналу и задержкам.
3. **Куда вставлять:** раздел о границах применимости layer-split.
4. **Ограничение:** результаты зависят от конкретной архитектуры и сетапа.

22.4 D.4 Бенчмарки и датасеты

22.4.1 [R13] MLPerf Tiny

1. **Суть:** стандартизованный benchmark suite для tiny inference.
2. **Тезис:** без единого протокола измерения сравнения малоценны.
3. **Куда вставлять:** раздел о методологии эксперимента.
4. **Ограничение:** твоя прикладная задача может выходить за стандартный набор.

22.4.2 [R24] Speech Commands

1. **Суть:** референсный датасет для limited-vocabulary KWS.
2. **Тезис:** обеспечивает baseline воспроизводимости.
3. **Куда вставлять:** описание датасетов.
4. **Ограничение:** «чистый» датасет, нужна доменная адаптация к бытовому шуму.

22.4.3 [R31] Temporal Lambda Networks for KWS

1. **Суть:** более эффективное моделирование дальних зависимостей для KWS.
2. **Тезис:** можно рассмотреть как teacher или как сравнение архитектур.
3. **Куда вставлять:** раздел выбора архитектур.
4. **Ограничение:** может быть тяжеловато для жестких MCU budget.

22.4.4 [R32] MobileNets

1. **Суть:** depthwise separable свертки как база efficient CNN.
2. **Тезис:** архитектурная философия mobile/edge моделей.
3. **Куда вставлять:** вводный раздел по compact networks.
4. **Ограничение:** не специфично для распределенной системы.

22.5 D.5 Официальные источники Espressif (обязательны в дипломе)

22.5.1 [R21] ESP-NOW API

1. **Суть:** формат, ограничения пакетов, callback semantics, peer management.
2. **Тезис:** сетевой протокол не «прозрачный», есть строгие инженерные требования.
3. **Куда вставлять:** глава реализации коммуникации.

22.5.2 [R22] Wi-Fi performance/power-save

1. **Суть:** throughput ранги, буферные конфиги, power-save режимы.
2. **Тезис:** задержка зависит не только от модели, но и от сетевых настроек.
3. **Куда вставлять:** раздел latency modeling и tuning.

22.5.3 [R20] FreeRTOS SMP в ESP-IDF

1. **Суть:** pinning, scheduler behavior, tick, critical sections.
2. **Тезис:** архитектура задач критична для стабильного realtime.
3. **Куда вставлять:** глава по firmware architecture.

22.5.4 [R19] External RAM / PSRAM

1. **Суть:** как использовать PSRAM и какие ограничения по производительности.
2. **Тезис:** PSRAM расширяет объем, но не гарантирует low latency.
3. **Куда вставлять:** раздел memory design.

22.5.5 [R16], [R17], [R23], [R25]

1. **Суть:** официальный toolchain Espressif для TinyML deployment.
2. **Тезис:** проект должен строиться на поддерживаемом стеке, иначе высок риск нестабильности.
3. **Куда вставлять:** практическая реализация и reproducibility.

23 Приложение Е. Сильные и слабые аргументы на защите

23.1 Е.1 Сильные аргументы

1. Тема строго соответствует формулировке диплома: распределенная сеть нейросетей на MCU.
2. Есть измеримый реальный эффект в бытовой задаче.
3. Есть научная составляющая через новую функцию потерь/маршрутизации.
4. Есть реплицируемая методология и baseline-сравнение.

23.2 Е.2 Слабые аргументы (которые лучше не использовать)

1. «Мы сделали сложную систему, значит это инновация».
2. «На одном наборе данных получилось лучше, значит везде лучше».
3. «Средняя задержка маленькая, значит realtime выполнен».

23.3 Е.3 Что спросит комиссия и как отвечать

Вопрос:

1. Почему не один более мощный узел вместо нескольких?

Ответ:

1. Многопозиционный сбор сигнала повышает робастность к локальному шуму и экранированию.
2. Распределенная архитектура снижает single point of failure.

Вопрос:

1. Где научная новизна, а не просто инженерный прототип?

Ответ:

1. В формализации communication-aware distillation и latency-aware routing.
2. В экспериментальном доказательстве при packet loss и бытовых шумовых сценариях.

Вопрос:

1. Почему не сделать «чистый split по слоям»?

Ответ:

1. Для ESP-класса устройств это часто проигрывает из-за стоимости передачи активаций.
2. Мы это не игнорируем, а показываем как baseline/контрпример.

24 Приложение F. Развернутые обзоры статей (сплошной текст)

Этот раздел добавлен как «вторая глубина» литературного обзора. Здесь статьи разбираются не в формате коротких тезисов, а как связанная исследовательская линия: что именно было сделано, почему это важно для твоей темы и где в этих работах остаются незакрытые вопросы.

24.1 F.1 TinyML и MCU-рантаймы

Работа CMSIS-NN [R1] важна тем, что она сделала обсуждение TinyML «земным». До нее многие разговоры о нейросетях на микроконтроллерах были скорее концептуальными, а после нее появилось ясное инженерное сообщение: на MCU судьбу инференса решают не только архитектура сети и число параметров, но и то, как именно реализованы базовые kernels под конкретную ISA и memory hierarchy. Авторы показывают кратные ускорения относительно наивной реализации и тем самым задают стандарт для всех последующих TinyML-стеков. Для твоего диплома это означает, что распределенная архитектура без низкоуровневого исполнения неубедительна: сначала должны быть эффективные кирпичики вычисления на каждом узле, и только потом стоит говорить о распределении блоков по сети.

MCUNet [R2] идет дальше и утверждает уже не «ускоряйте kernels», а «проектируйте модель и рантайм совместно». Это очень важный методологический поворот. Авторы показывают, что поиск архитектуры в абстракции «все устройства одинаковые» плохо работает для MCU-класса, где ограничения памяти и буферов фундаментально другие. Связка TinyNAS + TinyEngine демонстрирует, что правильная сеть для микроконтроллера часто отличается

от той, что казалась оптимальной в обычной mobile-среде. Для твоей задачи это прямой мост к распределенной постановке: если даже один MCU требует co-design, то сеть из нескольких MCU тем более требует co-design уже на уровне «модель + размещение + протокол».

В MCUNetV2 [R3] подробно разобран феномен memory peak в ранних блоках CNN. Это не «мелкая оптимизация», а структурная причина, почему многие модели формально помещаются по весам, но не проходят по оперативной памяти на реальном инференсе. Предложенное patch-based расписание инференса уменьшает пиковую память и расширяет пространство исполнимых архитектур. Для твоего диплома ценность этой работы в том, что она учит смотреть на память как на временной профиль, а не на одну итоговую цифру. Когда ты распределяешь блоки по узлам, профиль памяти каждого узла становится ограничением не менее важным, чем радиоканал.

On-Device Training Under 256KB Memory [R4] часто неправильно интерпретируют как «доказательство, что обучение на MCU уже решено». На самом деле статья показывает скорее границу возможностей и цену ее достижения: авторы добиваются обучения в очень жестком бюджете за счет глубокой algorithm-system кооптимизации и специальных упрощений backprop. Для твоего диплома это не основная траектория, но важный аргумент для раздела будущих работ. Если ты захочешь добавить персонализацию в распределенный нейрограф, надо сразу признавать: это отдельный сложный трек, который не заменяет задачу надежного распределенного инференса.

MEMA Runtime [R5] добавляет еще один существенный слой: стоимость доступа к внешней памяти. В TinyML-пайплайнах часто недооценивается тот факт, что перенос буферов во внешнюю память может спасать по объему, но резко ухудшать время. Для ESP32-семейства с PSRAM это наблюдение особенно практично. Статья систематизирует, как минимизировать внешние обращения при ограниченных ресурсах, и хорошо дополняет MCUNet-подход: даже правильная архитектура сети теряет преимущество, если рантайм провоцирует избыточный memory traffic.

24.2 F.2 Компрессия, квантование и перенос знаний

Классическая работа по **integer-only quantization** [R6] остается опорной потому, что предлагает не просто постфактум сжатие, а согласованную схему обучения и инференса в целочисленной арифметике. Для твоего проекта это важно в двух местах. Во-первых, это базовый путь получить стабильный и воспроизводимый инференс на ESP32 без плавающих затрат. Во-вторых, если в распределенной схеме между блоками передаются токены, то их квантование должно проектироваться так же аккуратно, как квантование самой модели; иначе коммуникационный выигрыш съедается падением качества.

Работа **Distilling the Knowledge in a Neural Network** [R7] задает фундаментальный механизм, который в твоей теме особенно полезен: учить легкие узловые модели имитировать поведение более сильного teacher, сохраняя компактность. Но в распределенной постановке дистилляцию выгодно переосмыслить: student должен учиться не только «угадывать правильный класс», но и формировать такие промежуточные представления, которые дешевы в передаче и полезны для соседних блоков. Именно из этого вырастает твой потенциальный вклад communication-aware distillation.

Эмпирическая работа о low-precision для TinyML [R8] ценна тем, что она убирает опасную иллюзию «чем меньше бит, тем лучше». В реальных задачах и на реальном железе оптимум редко лежит на самом агрессивном сжатии. Возникает баланс между точностью, устойчивостью, распределением ошибок по классам и аппаратными ограничениями. Для диплома это полезно методологически: ты можешь обосновать выбор INT8/смешанных стратегий как экспериментально подтвержденный компромисс, а не как догму.

MobileNets [R32] и похожие efficient-архитектуры важны уже как «фон архитектурной культуры». Их вклад не в распределенность как таковую, а в то, что они делают возможным перенос значимых CNN-задач в область ограниченных устройств за счет depthwise separable design и явного контроля ширины/разрешения модели. В твоей работе MobileNet-парадигма полезна как источник блоков-кандидатов для построения распределенного нейрографа.

24.3 F.3 Distributed и split inference: эволюция идей

Хотя **Neurosurgeon** [R40] не писался про микроконтроллерные mesh-сети, он исторически важен как одна из первых системных формализаций «где резать DNN между устройством и сервером». Его ключевая идея — использовать профилирование слоев и сетевых условий, чтобы выбирать точку разреза, минимизирующую заданную стоимость (задержку или энергию). Для твоего диплома это фундаментальный предшественник в логике оптимизации разбиения. Слабое место, которое ты можешь закрыть, состоит в том, что модель там в основном device-cloud, а у тебя — MCU-cluster с более жестким каналом и более выраженным jitter.

Работа **Edgent** [R10] усиливает это направление, добавляя адаптивность и early-exit логику. Она показывает, что в изменяющемся канале «одна фиксированная точка разреза» часто проигрывает. Именно эта идея хорошо переносится на твой кейс: блочная сеть на ESP32 должна поддерживать режимы исполнения с разной глубиной и разным

сетевым следом, иначе она хрупка к реальным условиям связи.

Очень близкой к твоей исходной гипотезе является работа **DDNN (Distributed Deep Neural Networks over the Cloud, the Edge and End Devices)** [R33]. В ней явным образом появляется многоуровневая распределенная архитектура, где разные части сети работают на разных уровнях иерархии. Статья особенно ценна тем, что обсуждает не только latency, но и сенсорную кооперацию и устойчивость. Для твоей темы это сильный аргумент: «блочная» мысль уже научно легитимна, и вопрос теперь в том, как ее адаптировать под гораздо более ограниченный класс узлов.

NoNN [R11] делает следующий шаг: ставит memory и communication awareness в центр декомпозиции teacher-модели. Это почти прямой теоретический каркас для твоего диплома, потому что проблема «в один MCU не помещается» у вас одинаковая, как и необходимость разложить модель на части с контролем трафика. Ограничение NoNN с точки зрения практики ESP32 в том, что paper-level декомпозиция нужно доводить до уровня конкретного протокола, очередей и типов буферов.

Линия **BottleNet** [R34] и **BottleFit** [R35] принципиально важна для «спасения» layer-split. Эти работы показывают: если вставить обучаемые bottleneck-представления в точки разреза и обучать сеть с учетом сжатия промежуточных признаков, можно радикально уменьшить объем передачи и сохранить качество. Именно это является ключом к твоей идее. На ESP32 смысл ровно такой же: без bottleneck-адаптеров межузловая передача активаций слишком дорога, с адаптерами она превращается в управляемый бюджет.

deepFogGuard [R36] добавляет то, что часто забывают в «красивых» split-диаграммах: отказоустойчивость. Если один физический узел выпал, обычный распределенный DNN может деградировать резко и непредсказуемо. Предложение skip-связей и обучения с учетом отказов хорошо ложится на микроконтроллерные кластеры, где выпадение узлов и потери пакетов — нормальный режим, а не исключение.

Scission [R37] полезен как системная работа про контекстно-зависимое распределение DNN между end/edge/cloud. Основной практический урок в том, что оптимальная конфигурация не статична: она зависит от текущей нагрузки и доступных ресурсов. Для твоего диплома это прямое обоснование runtime-policy, которая выбирает маршрут по графу блоков в зависимости от условий канала и состояния узлов.

DEFER [R38] делает акцент на distributed edge inference ради throughput и разгрузки отдельных устройств. Хотя у тебя может быть другой первичный KPI (например, tail-latency или energy/event), DEFER важен тем, что показывает реализуемость partition + placement на нескольких edge-устройствах как инженерной системы, а не только как оффлайн-оптимизации.

Работа по **partitioning and deployment on edge clusters** [R39] дополняет картину аспектом orchestration и масштабирования. Авторы формулируют задачу минимизации bottleneck latency при размещении частей модели на кластере ограниченных устройств. Для твоего проекта это особенно полезно, если ты хочешь показать, что архитектура не ломается при переходе от 2–3 узлов к 5–8 узлам.

Свежий **split-learning TinyML testbed на ESP32-S3** [R12] ценен как мост между теорией и твоим целевым железом. Он подтверждает, что тема не «архивная», а активно развивающаяся именно на class of devices, который тебя интересует. Важный урок из этой ветки — необходимость измерять не абстрактную вычислительную сложность, а полный end-to-end путь с реальной радиопередачей.

Наконец, **SpikeBottleNet** [R41] интересен как новый взгляд на коммуникационное сжатие через событийные представления. Даже если ты не будешь использовать SNN напрямую, статья показывает направление: промежуточные признаки можно проектировать так, чтобы они были информативны для следующего блока и одновременно дешевы для передачи. Это полностью совпадает с логикой CC-NBG.

24.4 F.4 Бенчмарки, датасеты и прикладные ориентиры

MLPerf Tiny [R13] для твоей работы важен прежде всего как методология, а не как «обязательный тест». Он заставляет фиксировать протокол измерений, условия запуска, метрики latency/energy и правила сравнения. Это снимает частую критику дипломных прототипов: «результат есть, но воспроизвести его нельзя». Даже если твоя распределенная задача выходит за стандартный набор MLPerf Tiny, сама культура benchmark-отчетности оттуда должна быть перенесена в проект почти без изменений.

Speech Commands [R24] и статьи по KWS на MCU [R9][R31] остаются полезными, но в текущей версии отчета их роль изменена. Они используются не как «смысл диплома», а как доступный тест-домен для проверки универсальной архитектурной идеи. Это важное позиционирование на защите: предметная задача может быть акустической, но научный объект — распределенный нейрограф на микроконтроллерах, который переносим на другие сенсорные модальности.

Статья **Widening Access to Applied Machine Learning with TinyML** [R30] добавляет полезный контекст «зачем это вообще нужно». Она показывает, что TinyML интересен не только академически, но и как инженерный путь к массовым недорогим системам, где локальный инференс и автономность важнее облачной мощности. Для диплома это хороший аргумент в мотивационной главе: тема не узкая, а стратегически релевантная.

25 Приложение G. Финальная архитектурная позиция после критики

После анализа обоих отчетов и замечаний критика можно зафиксировать окончательную мысль так: противопоставление «CC-NBG против MoE» в чистом виде неверно. Для ESP32-кластера наиболее жизнеспособна гибридная архитектура, в которой базовый режим — это MoE/decision-level кооперация с короткими сообщениями, а блочный split с bottleneck-токенами включается только как условный второй путь для сложных случаев.

Это решение сохраняет фундаментальность темы «конфигурируемой сети блоков на микроконтроллерах», но избегает главной практической ловушки наивного layer-split: жесткой зависимости всей цепочки от своевременной доставки крупных промежуточных тензоров.

Критик прав в главном: если разрезать сеть без коммуникационного дизайна, система действительно упирается в канал и джиттер Wi-Fi стека. Однако тезис «всегда проще считать на одном чипе» слишком сильный и неуниверсальный. Он работает только там, где модель помещается, хватает вычислительного запаса и не требуется межузловая специализация. В распределенной постановке, где нужны конфигурируемость, устойчивость и масштабирование по функциям, один чип перестает быть эквивалентной альтернативой.

25.1 G.1 Конкретный критерий жизнеспособности split с энкодером/декодером

Для локального full-inference:

$$T_{local} = T_{full}$$

Для split-пути через bottleneck:

$$T_{split} = T_{head} + T_{enc} + T_{net} + T_{dec} + T_{tail} + T_{sync}$$

Split имеет смысл только если одновременно выполнены условия:

$$T_{split,p95} < T_{local,p95},$$

$$\Delta Acc \leq \epsilon,$$

$$B_{token} \leq B_{budget}.$$

Здесь и находится ответ на вопрос «почему нельзя просто добавить энкодер/декодер». Можно, и это правильная идея в духе BottleNet/BottleFit/SpikeBottleNet [R34][R35][R41], но только если энкодер/декодер учатся в составе end-to-end цели и если их вычислительная цена меньше выигрыша от уменьшения сетевой передачи. Иначе компрессор превращается в дорогой фильтр, который ухудшает и latency, и качество.

25.2 G.2 Практическое решение для диплома: гибридный двухконтурный режим

Финальная рекомендуемая архитектура следующая. Первый контур — MoE-fast path. Узлы-специалисты дают локальные logits/оценки, координатор делает routing и fusion. Этот контур устойчив, прост в отладке и хорошо переносит packet loss, потому что каждый узел — законченный инференс-эксперт, а не половина чужого слоя.

Второй контур — CC-NBG bottleneck path. Он запускается только при высокой неопределенности или запросе уточнения. В нем используются заранее обученные блоки split-инференса с токенами строго ограниченного размера. Это и есть «спасенная» блочная идея в научно корректной форме.

Такой гибрид дает три преимущества одновременно: малую базовую задержку, управляемый трафик и наличие полноценно распределенного нейросетевого графа, который можно конфигурировать и исследовать как фундаментальную архитектуру.

25.3 G.3 Что именно будет научной новизной в такой финальной версии

Новизна не в самом факте «мы запустили несколько ESP32», а в методе управления компромиссами: совместная оптимизация accuracy/latency/traffic для распределенного нейрографа, адаптивный выбор контура исполнения по confidence и состоянию канала, формализованный критерий включения split-пути с bottleneck-токеном, и экспериментально подтвержденная устойчивость при потере пакетов/узлов.

Именно этот набор делает работу исследовательской, а не просто интеграционным демо.

25.4 G.4 Жесткая практическая конкретика

Токены межузлового обмена должны оставаться маленькими и фиксированными по формату (ориентир: десятки байт, а не килобайты). Нужно мерить не только среднюю, а именно p95/p99 задержку и packet-loss sensitivity как основной критерий сетевой жизнеспособности [R21][R22]. Split-путь не должен быть обязательным шагом каждого инференса: он включается условно. Любой bottleneck-адаптер обучается end-to-end вместе с задачей, а не как отдельный «внешний автоэнкодер». В отчете должны быть обе зоны: где гибрид выигрывает, и где честно проигрывает single-node режиму.

Если эта дисциплина соблюдается, исходная идея с блоками не просто «спасается», а становится сильной центральной осью диплома.